

Activity 1

Create hash values

As a security analyst, I can implement security control to protect organisations against a range of threats.

Here in this scenario, I was given a task to investigate whether two files are identical or different.

Task 1. Generate hashes for files

I started the lab in my home directory.

Step 1

First, I ran ***ls*** command to display the contents of the directory. I noticed that there were two files (***file1.txt*** and ***file2.txt***)

Next, I ran ***cat file1.txt*** and ***cat file2.txt*** command to display the contents of both files. At this point, the contents of both files appeared identical.

Then I used ***sha256sum*** command to generate the hash of both files by using following commands:

Shah256sum file1.txt

Sha256sum file2.txt

Here, one thing is noticeable that the generated hash value of file1.txt is different from file2.txt.

Task 2. Compare hashes

Here I wrote the hashes to two separate files and then compared them to find difference.

I ran ***sha256sum file1.txt >> file1hash*** and ***sha256sum file2.txt >> file2hash*** commands to generate the hash of both files and sent the output to new files ***file1hash*** and ***file2hash***.

Next, I used ***cat*** command to display the hash values for both files.

Cat file1hash

Cat file2hash

Finally, I used ***cmp*** command to compare the difference

cmp file1hash file2hash

```

analyst@7d0a8e912ff7:~$ /home/analyst
-bash: /home/analyst: Is a directory
analyst@7d0a8e912ff7:~$ ls
file1.txt  file2.txt
analyst@7d0a8e912ff7:~$ cat file1.txt
X50!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
analyst@7d0a8e912ff7:~$ cat file2.txt
X50!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
9sxa5Yq20Ranalyst@7d0a8e912ff7:~$ sha256sum file1.txt
131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbfd8267  file1.
txt
analyst@7d0a8e912ff7:~$ sha256sum file2.txt
2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.
txt
analyst@7d0a8e912ff7:~$ sha256sum file1.txt >> filehash
analyst@7d0a8e912ff7:~$ sha256sum file1.txt >> file1hash
analyst@7d0a8e912ff7:~$ sha256sum file2.txt >> file2hash
analyst@7d0a8e912ff7:~$ cat file1hash
131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbfd8267  file1.
txt
analyst@7d0a8e912ff7:~$ cat file2hash
2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.
txt
analyst@7d0a8e912ff7:~$ cmp file1hash file2hash
file1hash file2hash differ: char 1, line 1
analyst@7d0a8e912ff7:~$ █

```

The result shows that the contents of both files are different as the hash values are different.

Challenge: Sometimes, we may misspell or type any command wrong, but without shutting the shell completely down, we can take close look to see if that can be solved. Here, I forgot to type **file1.txt**. However, I knew that I am not working with file, I am working with file1 and file2. So, I kept going.

Conclusion:

The commands: shas256sum, cat, cmp –all are valuable tools for a security analyst as these will be used to validate data integrity to contribute the organisation’s security on regular base.

Activity 2 : Decrypt an encrypted message

Scenario

I was told that all the files in my home directory were encrypted. I had to use Linux commands to break the Caesar cipher and decrypt the file so that I could read the hidden messages from them.

Step 1

I used **ls** command to list the files in current directory. Then, I ran a query using **cat** command to display what README.txt file contains and if there is any other subdirectory.

Step 2

I used **cd caesar** command to change to the caesar subdirectory of my home directory. Then, I ran **ls -a** command to see if there are any hidden files. In this case, I found **.leftShift3**. Because I know **.** indicates hidden file. Next, I displayed this file using **cat .leftShift3** command. Then I ran command **cat .leftShift3 | tr "d-za-cD-ZA-Z" "a-zA-Z"** where I am telling to translate (tr) everything in their normal position. Next, I used **cd ~** command to return to my home directory.

```
analyst@7fa4e991e833:~$ ls
Q1.encrypted  README.txt  caesar
analyst@7fa4e991e833:~$ cat README.txt
Hello,
All of your data has been encrypted. To recover your data, you will need
to solve a cipher. To get started look for a hidden file in the caesar
subdirectory.
analyst@7fa4e991e833:~$ cd caesar
analyst@7fa4e991e833:~/caesar$ ls -a
.  ..  .leftShift3
analyst@7fa4e991e833:~/caesar$ cat .leftShift3
Lq rughu wr uhfryhu brxu ilohv brx zloo qhhg wr hqwuhu wkh iroorzlqj frpp
dgg:

rshqvvo dhv-256-fef -sengi2 -d -g -lq T1.hqfubswhg -rxw T1.uhfryhuhg -n
hwwxeuxwh
analyst@7fa4e991e833:~/caesar$ cat .leftShift3 | tr "d-za-cD-ZA-C" "a-zA
-Z"
In order to recover your files you will need to enter the following comm
and:

openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k
ettubrute
```

Step 3

After solving the caesar cipher, I used the command revealed in step 2 which was

openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k ettubrute

Here, **-256** is hashing algorithm, **-pbkdf2** is public key, **-a** indicates the desired encoding for the output, **-d** indicates decrypting, **-out** specifies the output file, **-k** specifies the password (ettubrute).

Next, I ran the **ls** command again. This time it's showing file as '**Q1.recovered**'. Finally, I ran **cat Q1.recovered** command to display the content of the file.

```
analyst@7fa4e991e833:~/caesar$ cd ~
analyst@7fa4e991e833:~$ openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k ettubrute
analyst@7fa4e991e833:~$ ls
Q1.encrypted  Q1.recovered  README.txt  caesar
analyst@7fa4e991e833:~$ cat Q1.recovered
If you are able to read this, then you have successfully decrypted the classic cipher text. You recovered the encryption key that was used to encrypt this file. Great work!
analyst@7fa4e991e833:~$
```

Conclusion:

As security analyst, I am prepared to encrypt and decrypt file to keep the organisational data safe.